
[C++] Klasa biblioteczna `std::string`

| Ksawery Zelek | 4TI | numer: 31 |

Dlaczego klasa `std::string` została wprowadzona?

Klasa `std::string` w języku C++ została wprowadzona, aby ułatwić manipulację napisami znakowymi. Wcześniej, w języku C, obsługa napisów była bardziej uciążliwa, ponieważ wymagała ręcznego zarządzania pamięcią i nie miała wbudowanych mechanizmów ułatwiających operacje na napisach. Klasy `string` w C++ miały na celu zautomatyzowanie wielu operacji na napisach, jednocześnie zapewniając bezpieczne zarządzanie pamięcią.



Więc dlaczego ją wprowadzili? - krótko

Pomysł na wprowadzenie tej klasy wynikał z potrzeby zapewnienia programistom prostego i efektywnego narzędzia do pracy z napisami, jednocześnie eliminując potencjalne pułapki związane z zarządzaniem pamięcią.

**00101011
01101010
101110101
11011000
10100110**

Jakie są zalety i wady tej klasy?

Zalety

- Dynamiczna długość i elastyczne zarządzanie pamięcią.
- Bezpieczeństwo operacji, eliminacja ryzyka przepełnienia bufora.
- Część standardowej biblioteki C++, co zapewnia przenośność kodu.
- Bogate wsparcie dla operacji na napisach.

Wady

- Nieco mniejsza efektywność w niektórych przypadkach.
 - Niemutowalność, co może prowadzić do dodatkowego zużycia pamięci.
 - Możliwość rzucania wyjątkami, zwłaszcza przy alokacji pamięci.
-

Przegląd konstruktorów klasy std::string

1. string()

W rezultacie powstaje obiekt klasy string, który nie jest niczym inicjalizowany, czyli (wstępnie) jest pusty.

```
string napisA;
```

2. string(const char * cstr)

Jest to konstruktor, którego argumentem jest wskaźnik do C-stringu. Oto jak na przykład można się posłużyć tym konstruktorem.

```
string napisB1( "Jakis tekst" );  
  
// Albo  
  
char tablica[ 20 ] = { "Natenczas Wojski" };  
string napisB2( tablica );
```

Przegląd konstruktorów klasy std::string

3. string(const char * cstr, size_t ile)

→ ile: jest argumentem określającym, ile znaków konstruktor ma przeczytać,

→ cstr: wskaźnik do c-str

```
string ostrzezenie( "Awaria studni", 8 );  
  
// ostrzezenie = "Awaria s"
```

4. string(size_t ile, char znak)

→ znak: argument typu char, określający znak, który ma być wielokrotnie powtarzany

→ ile: liczba określająca, jak długi ma być tekst

```
string str( 3, 'a' );  
  
// str = "aaa"
```

Przegląd konstruktorów klasy std::string

5. `string( const string& inny_string, size_t pozycja, size_t ile )`

Konstruktor ten ze wskazanego mu innego obiektu klasy string bierze znaki począwszy od wskazanej pozycji (pozycje numerujemy od 0). Liczbę pobranych w sumie znaków określa argument ile.

```
string inny_string = "ABCDEFGH";  
string nowy( inny_string, 4, 2 );  
  
// nowy = "EF"
```

Czy można dodawać/odejmować stringi?

W języku C++, stringi można dodawać, łącząc je za pomocą operatora +, a także modyfikować poprzez usuwanie i zmianę fragmentów. Odejmnowanie stringów nie ma standardowego znaczenia, ale można usunąć lub zamienić fragment napisu. W przypadku tradycyjnych C-stringów, operacje manipulacji, takie jak konkatencja (strcat) lub zamiana (strcpy), również są dostępne.

—

NAJWAŻNIEJSZE FUNKCJE SKŁADOWE

`size_t string::size()`

Pokazuje, ile znaków jest w zmiennej

`size_t string::length()`

Pokazuje, ile znaków jest w zmiennej

`size_t capacity()`

Pokazuje, jak duża jest bieżąca rezerwacja miejsca

`string substr( size_t p, size_t n = string::npos )`

Funkcja wydobywa z głównego stringu inny podstring

`size_t find( “jest kilka opcji” )`

Poszukuje podane znaki

```
const char* c_str()
```

udostępnia treść klasy string w postaci C-stringu

```
[...] .at( size_t poz )
```

Odnosi się w danym stringu do wybranego arg. **poz**

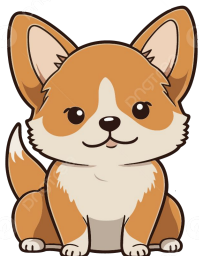
```
void clear()
```

Kasuje dotychczasową treść obiektu

—

Przykładowe użycie klasy `std::string` 2 przykłady

Klasa zwierze



```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Tzwierze
6  {
7  public:
8      Tzwierze( const std::string imie = "" ) : imie( imie )
9      { }
10
11      // Pobieranie imienia
12      std::string pobierzImie() const
13      {
14          return imie;
15      }
16
17  private:
18      std::string imie;
19  };
```

Klasa Figura Geometryczna



```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class TfiguraGeometryczna          // Kwadrat
6  {
7  public:
8      TfiguraGeometryczna( double dl_boku ) : dl_boku( dl_boku )
9      { }
10
11      // Metoda, do obliczania pola figury
12      double obliczPole() const
13      {
14          return dl_boku;
15      }
16
17  private:
18      double dl_boku;
19  };
```

DZIĘKUJĘ ZA UWAGĘ
